Solution_Lecture1

July 13, 2023

1. Print the result of the following operations using python:
   - $5 + 4 + 3 + 2 + 1$
   - $5 * 4 * 3 * 2 * 1$
   - 6^4

[1]:
```
print(5+4+3+2+1)
print(5*4*3*2*1)
print(6**4)
```

```
15
120
1296
```

2. Use the math package to compute the following expression
   - $e^{\pi} - 1$

[2]:
```
import math
y = math.exp(math.pi)-1
print(y)
```

```
22.140692632779267
```

[3]:
```
# alternative solution
from math import exp, pi
print(exp(pi)-1)
```

```
22.140692632779267
```

3. Let `a=True + True`. what is the type of the variable `a`?

[4]:
```
a=True+True
print('a=',a)
print('type of a=', type(a))
```

```
a= 2
type of a= <class 'int'>
```

4. Let `a=[1,2]` and `b = [5,1]` be two lists. what is the result of `c=3*a + b`?

[5]:
```
a, b = [1,2], [5,1]
c=3*a+b
print(c)
```

```
[1, 2, 1, 2, 1, 2, 5, 1]
```

5. Let `a=(6,8,3,0)` be a tuple.

- write a program that prints the second and last element of this tuple
- transform the previous tuple to a list and store the result in a variable called `b`.
- Insert the value `"A"` in the second position of the list `b`
- Delete the value `8` from the list
- Delete the first value of the list
- print the resulting list

[6]:
```
a=(6,8,3,0)

#1
print(a[1], a[-1])

#2
b = list(a)
print(b)

#3
b.insert(1,'A')
print(b)

#4
b.remove(8)
print(b)

#5
del b[0]

#6
print(b)
```

```
8 0
[6, 8, 3, 0]
[6, 'A', 8, 3, 0]
[6, 'A', 3, 0]
['A', 3, 0]
```

6. Use pandas to open the three files contained in the data folder of this lesson (`data_cymbals.csv`, `data_invoices.xlsx`, and `sales_us.txt`).

[7]:
```
import pandas as pd

#cymbals.csv
df1 = pd.read_csv('data_cymbals.csv')
df1.head(2)
```

[7]:
```
   day_1  day_2  day_3  day_4  day_5  day_6  day_7  day_8  day_9  day_10  …  \
0  440.0  440.0  439.6  440.6  440.1  439.4  438.7  439.9  440.9   439.7  …
1  440.1  440.0  439.6  439.2  439.9  440.8  440.0  439.9  440.7   440.4  …

   day_356  day_357  day_358  day_359  day_360  day_361  day_362  day_363  \
0    439.0    438.9    439.6    438.2    441.0    438.7    440.1    439.7
1    440.3    440.7    440.4    439.8    440.2    439.5    441.5    441.3

   day_364  day_365
0    438.9    439.8
1    440.7    441.9

[2 rows x 365 columns]
```

[8]:
```
# sales_us.txt
df2 = pd.read_csv('sales_us.txt', skiprows=3, sep=';')
df2.head(4)
```

[8]:
```
   Time  Boston   Portland
0     1   26.20  31.960473
1     2   24.37  26.943190
2     3   22.13  19.413672
3     4   23.94  21.188103
```

[9]:
```
# data_invoices.xlsx
df_invoices=pd.read_excel('data_invoices.xlsx', sheet_name='data_invoices')
df_invoices.head(2)
```

[9]:
```
   id_invoice  amount  age customer  items bought  credit card
0           1   163.5            43             7            0
1           2   138.8            39             5            0
```

[10]:
```
df_amount= pd.read_excel('data_invoices.xlsx', sheet_name='amount')
df_amount.head(2)
```

[10]:
```
   163.5
0  138.8
1  175.9
```

7. Create a numpy array, a, with the numbers from 0 to 6.

[11]:
```
import numpy as np
a = np.arange(0,7)
print(a)
```

```
[0 1 2 3 4 5 6]
```

8. Create a numpy array of length 10 with all the values equal to 8

[12]:
```
#method 1
a = 8*np.ones(10)
print(a)

#method 2
b = np.zeros(10) + 8
print(b)
```

```
[8. 8. 8. 8. 8. 8. 8. 8. 8. 8.]
[8. 8. 8. 8. 8. 8. 8. 8. 8. 8.]
```

9. Let `a=[1,2]` and `b = [5,1]` be two lists. Transform them to numpy arrays. What is now the result of `c=3*a + b`?

[13]:
```
a = [1,2]
b = [5,1]
a = np.array(a)
b = np.array(b)
c = 3*a+b
print(c)
```

```
[8 7]
```

10. Create an array of length 10 with the squares of the first natural numbers, that is 1,4,9,...

[14]:
```
a = np.arange(1,11)
squares = a*a
print(squares)
```

```
[  1   4   9  16  25  36  49  64  81 100]
```

Optional Exercises

1. If `a=5.3` is a numeric variable (for instance an integer); then `(a*9)/3 == a*(9/3)` is a Boolean expression with value True. However, if `a="A"`, a string, is the same expression `(a*9)/3 == a*(9/3)` still a correct Boolean statement?

[15]:
```
# test first claim
a=5.3
(a*9)/3 == a*(9/3)
```

[15]:
```
True
```

[16]:
```
# test second one
a="A"
(a*9)/3 == a*(9/3)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_27888\736245914.py in <module>
```

```
      1 # test second one
      2 a="A"
----> 3 (a*9)/3 == a*(9/3)

TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

It gives an error, and therefore it is NOT a valid boolean statement. The reason is that while ùultiplication of a string by an integer makes sense in python ("A"*8 = "AAAAAAAA"), the multiplication of a string by a fractional number (or the division) are not well defined.

2. Let `names=["one", "two", "three"]` and `numbers=[1,2,3]` two lists. Construct two dictionaries, one in which the names are the keys and the numbers the values, and another in which the numbers are the keys and the names are the values

[17]:
```
names=["one", "two", "three"]
numbers=[1,2,3]
dict_1 = {"one":1, "two":2, "three":3}
dict_2 = {1:"one", 2:"two", 3:"three"}
print(dict_1, dict_2)
```

```
{'one': 1, 'two': 2, 'three': 3} {1: 'one', 2: 'two', 3: 'three'}
```

3. Create a numpy array of 200 elements in which the fist 100 elements are ones; and the last 100 elements are zeros

Solution: we create first two arrays; one with 1 and another with the zeros, and then we will concatenate them.

[18]:
```
arr1 = np.ones(100)
arr2 = np.zeros(100)
arrFinal = np.concatenate([arr1, arr2])
print(arrFinal)
```

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0.]
```

Alternative solution. we create an array from 0 to 199, and we impose tht all number less than 100 ar zero, and all number bigger or equal to 100 go to 1.

[19]:
```
arr = np.arange(0,200)
arr[arr<100]=0
arr[arr>=100]=1
print(arr)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

4. Create another array, `b`, with all even numbers from 50 to 60 (included).

[20]:
```
arr = np.arange(50, 61, 2)
print(arr)
```

```
[50 52 54 56 58 60]
```

5. open the fime `cymbals.csv` with pandas. How many measurements in the column **day1** are less than 440?

[21]:
```
df1 = pd.read_csv('data_cymbals.csv')
df1.head(2)
```

[21]:

	day_1	day_2	day_3	day_4	day_5	day_6	day_7	day_8	day_9	day_10	…	\
0	440.0	440.0	439.6	440.6	440.1	439.4	438.7	439.9	440.9	439.7	…	
1	440.1	440.0	439.6	439.2	439.9	440.8	440.0	439.9	440.7	440.4	…	

	day_356	day_357	day_358	day_359	day_360	day_361	day_362	day_363	\
0	439.0	438.9	439.6	438.2	441.0	438.7	440.1	439.7	
1	440.3	440.7	440.4	439.8	440.2	439.5	441.5	441.3	

	day_364	day_365
0	438.9	439.8
1	440.7	441.9

```
[2 rows x 365 columns]
```

[22]:
```
data = df1['day_1']
data
```

[22]:
```
0     440.0
1     440.1
2     439.8
3     439.2
4     439.6
      …
95    440.3
96    440.6
97    440.0
98    440.0
99    439.5
Name: day_1, Length: 100, dtype: float64
```

I create a boolean array by comparing the data to the value 440. if the value in the daa array is less than 440, the boolean aray will be True, anf False otherwise. I can then sum all the elements on the array, because True and False behaves like 1 and 0 respectively when i apply a sum

[23]:
```
bool_arr = (data<440)
print(bool_arr)
```

```
0     False
1     False
2      True
3      True
4      True
      …
95    False
96    False
97    False
98    False
99     True
Name: day_1, Length: 100, dtype: bool
```

[24]:
```
n_meas_below_440 = sum(bool_arr)
print('there are ',n_meas_below_440,' measurement below 440 on day 1')
print('This coorrespond to', n_meas_below_440/len(data)*100,'% of the␣
 ↪measurements')
```

```
there are  46  measurement below 440 on day 1
This coorrespond to 46.0 % of the measurements
```